

Lecture-#7

Natural Language Processing

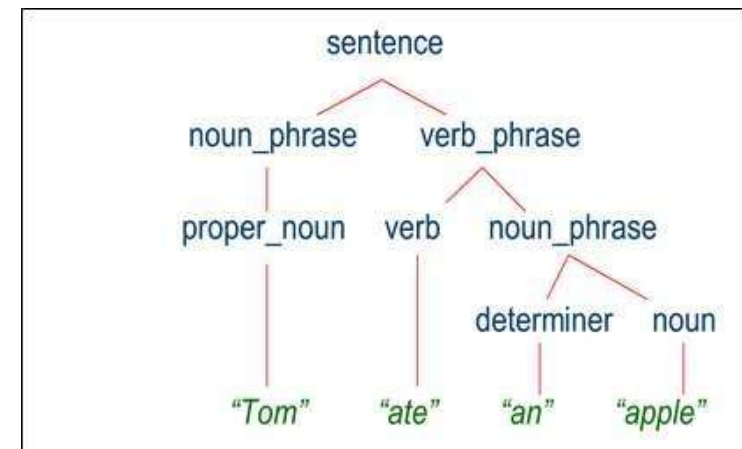
A8706

Syntax Analysis

Parsing Natural Language

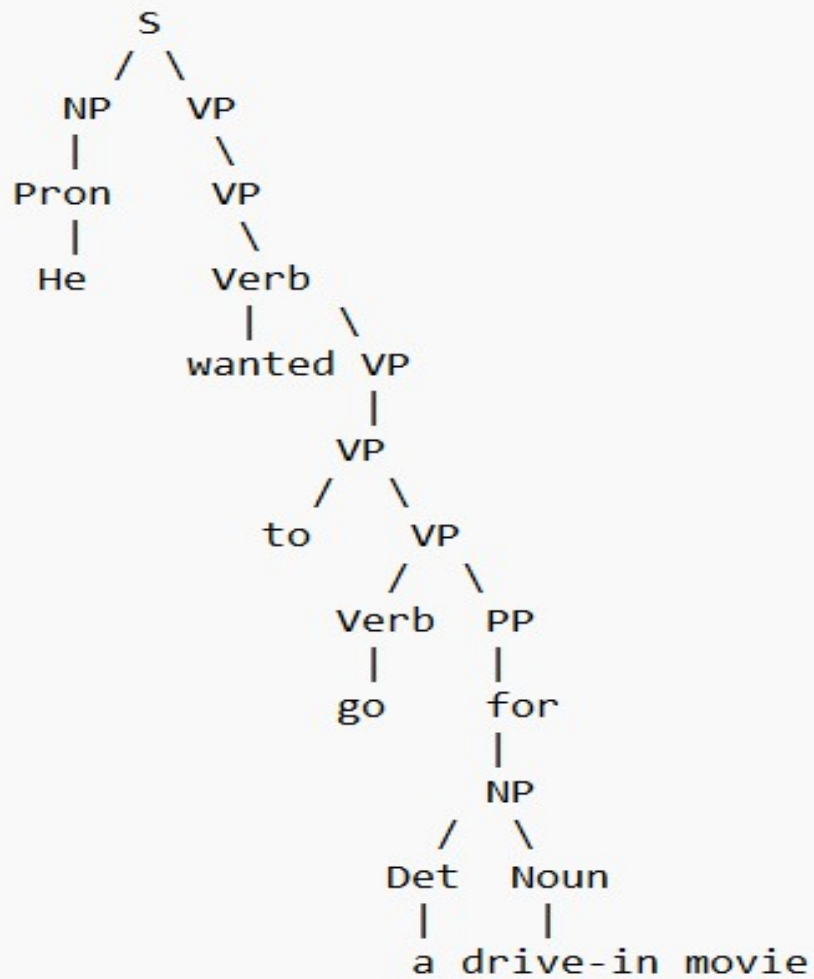
The parsing in NLP is the process of determining the syntactic structure of a text by analysing its constituent words based on an underlying grammar.

```
sentence -> noun_phrase, verb_phrase  
noun_phrase -> proper_noun  
noun_phrase -> determiner, noun  
verb_phrase -> verb, noun_phrase  
proper_noun -> [Tom]  
noun -> [apple]  
verb -> [ate]  
determiner -> [an]
```



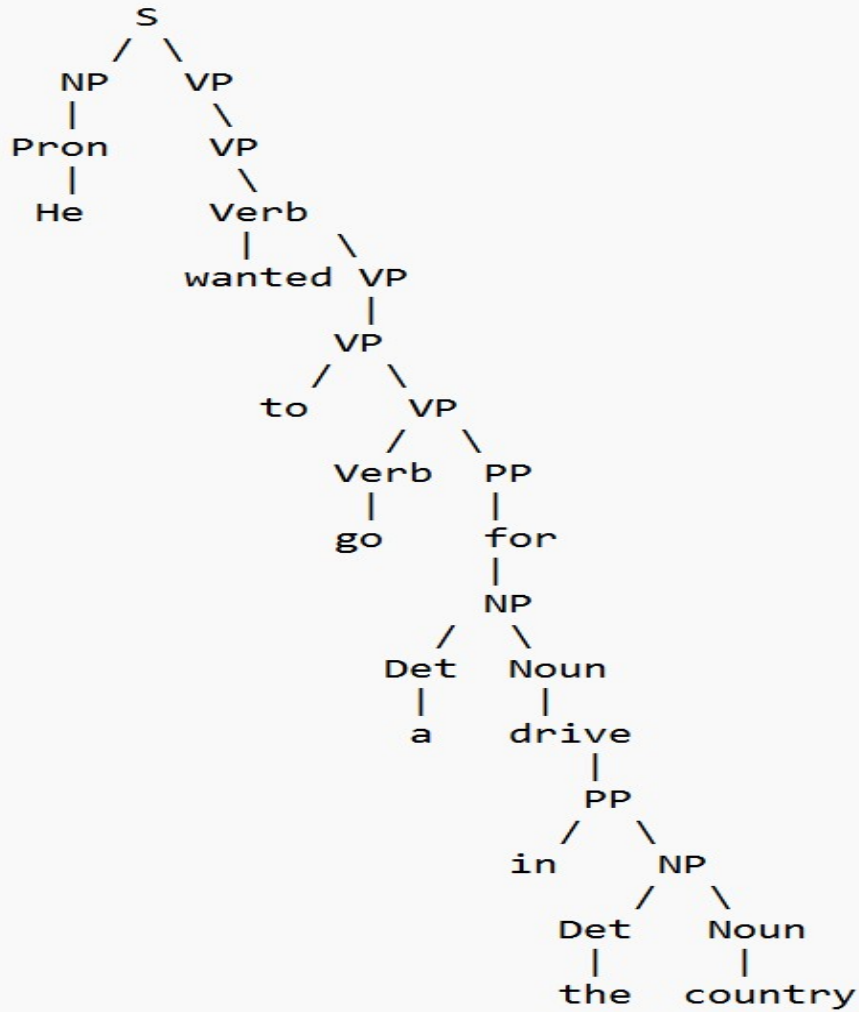
Then, the **outcome** of the parsing process would be a **parse tree**, where **sentence** is the root, intermediate nodes such as **noun_phrase**, **verb_phrase** etc. have children - hence they are called **non-terminals** and finally, the leaves of the tree '**Tom**', '**ate**', '**an**', '**apple**' are called **terminals**.

Parsing Natural Language



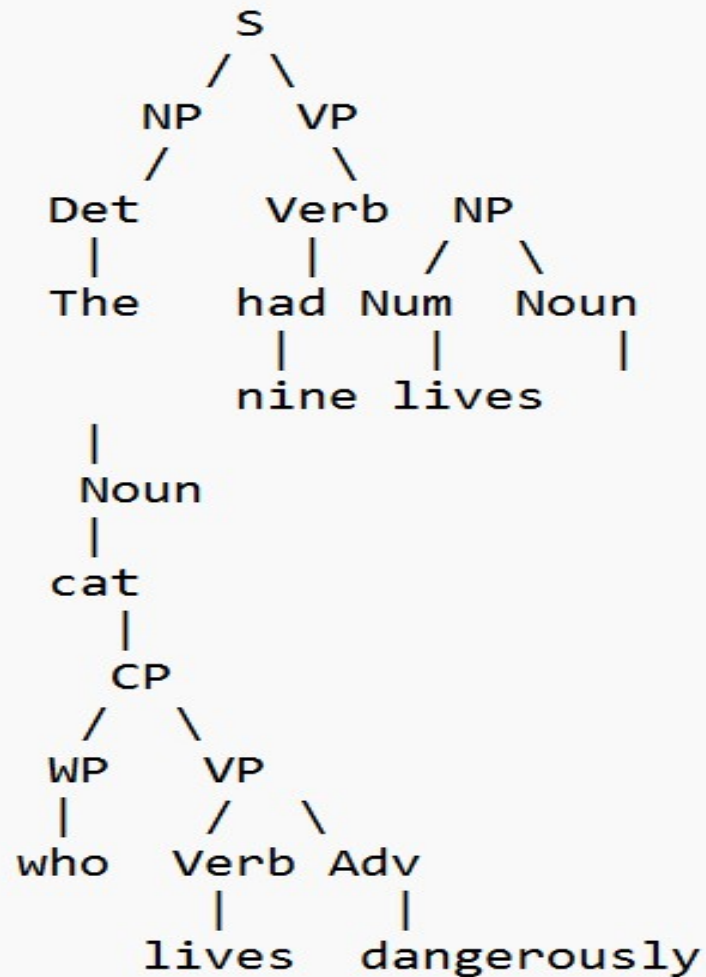
He wanted to go for a
drive in movie.

Parsing Natural Language



He wanted to go for a
drive in the country.

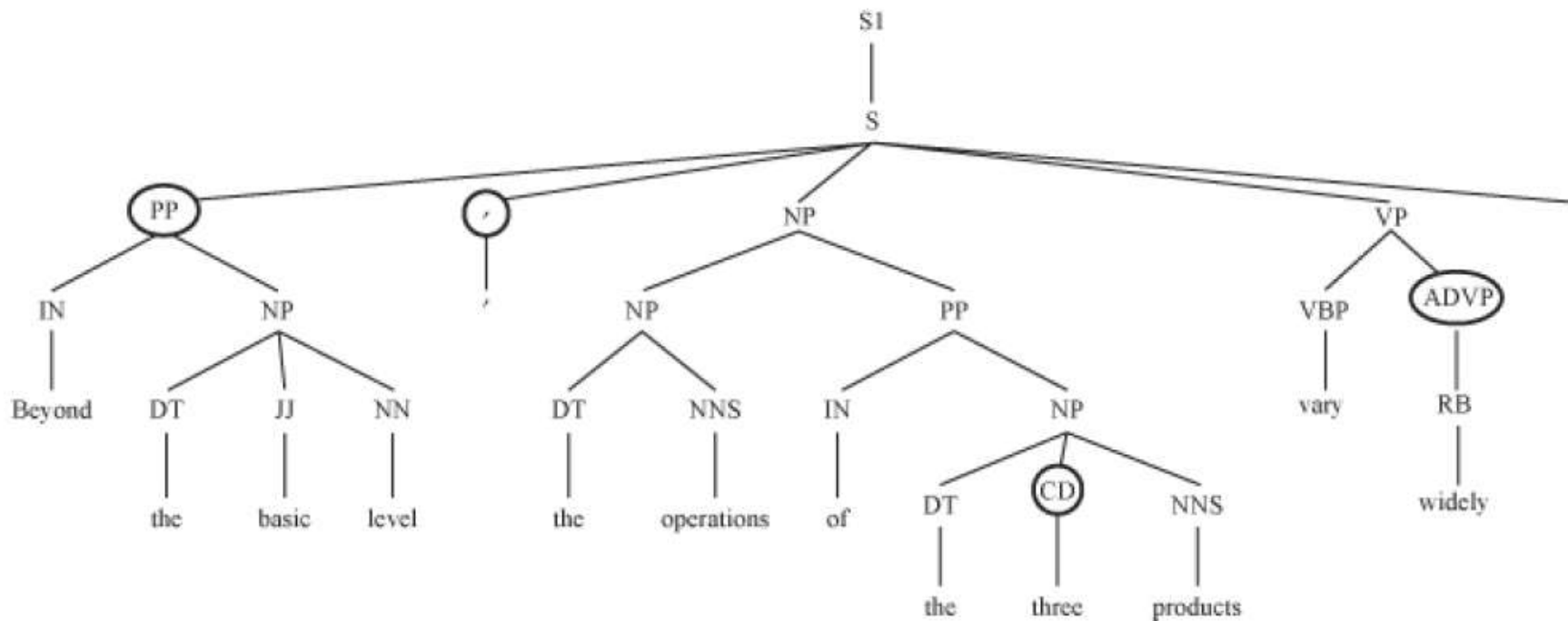
Parsing Natural Language



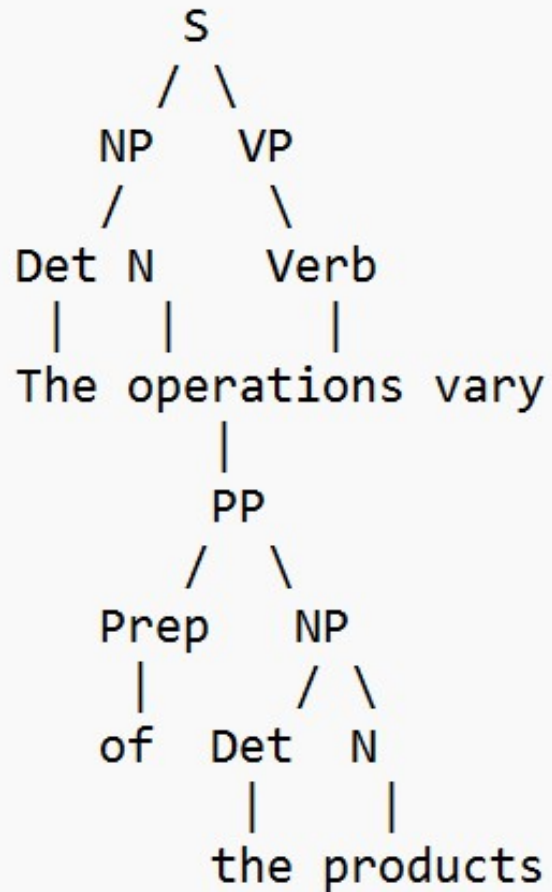
**The cat who lives
dangerously had nine lives.**

Parsing Natural Language

Beyond the basic level, the operations of the three products vary widely.



Parsing Natural Language



**The operations of the
products vary.**

Parsing Natural Language



Learning from Experience, Sharing with Passion

- ❖ A treebank can be defined as a linguistically annotated corpus that includes some kind of syntactic analysis over and above part-of-speech tagging.
- ❖ A sentence is parsed by relating each word to other words in the sentence which depend on it.
- ❖ The syntactic parsing of a sentence consists of finding the correct syntactic structure of that sentence in the given formalism/grammar.
- ❖ Dependency grammar (DG) and phrase structure grammar (PSG) are two such formalisms.
- ❖ PSG breaks sentence into constituents (phrases), which are then broken into smaller constituents.
- ❖ Describe phrase, clause structure Example: NP, PP, VP etc.,
- ❖ DG: syntactic structure consist of lexical items, linked by binary asymmetric relations called dependencies.
- ❖ Interested in grammatical relations between individual words.
- ❖ Does propose a recursive structure rather a network of relations
- ❖ These relations can also have labels.

Parsing Natural Language



Learning from Experience, Sharing with Passion

What Is a Treebank?

A treebank is a collection of sentences paired with detailed syntactic analyses—most often in the form of parse trees (constituent or dependency).

Why Use Treebanks?

- Provide **gold-standard** data for training and evaluating parsers
- Anchor and test **linguistic theories** of syntax
- Enable downstream NLP tasks (semantic role labeling, MT, etc.) to exploit syntax

Two Main Annotation Styles

1. Constituency (Phrase-Structure)

- Sentences broken into nested constituents (NP, VP, PP...)
- Commonly bracketed in Penn-Treebank style

2. Dependency

- Words linked by head–dependent relations
- Focuses on direct grammatical relationships (subject, object...)

Parsing Natural Language



Learning from Experience, Sharing with Passion

Key Treebank Projects

Penn Treebank (PTB)

- ~50,000 Wall Street Journal sentences
- Standard POS tags (NN, VBZ, DT...)
- Bracketed phrase-structure trees

Universal Dependencies (UD)

- Multilingual, harmonized POS tags and dependency labels
- Covers dozens of languages and genres

How a Treebank Is Built

- Text selection (newswire, fiction, web...)
- Automatic tokenization + POS-tagging
- Manual correction by trained annotators
- Tree annotation (constituent or dependency)
- Quality control and adjudication

Common Applications

- **Parser development:** train statistical or neural models
- **Information extraction:** improve relation and event extraction
- **Machine translation:** syntax-aware translation models
- **Linguistic research:** study syntactic patterns and constructions

Challenges & Limitations

- **High cost:** manual annotation is labor-intensive
- **Domain bias:** many treebanks focus on news text
- **Scheme variability:** different formats require conversion for combined use

Parsing Natural Language

```
(N (N (N natural)
      (N language))
 (N processing))
```

```
(N (N natural)
   (N (N language)
       (N processing)))
```

The Ambiguous Recursive Rule

- Suppose our grammar has (among others) this rule for stacking modifiers or nouns:
- $NP \rightarrow NP N$
and a base case like
- $NP \rightarrow \text{"book"} \mid \text{"processing"} \mid \text{"language"} \mid \text{"natural"}$
When you string together $k + 1$ base nouns ("natural language processing book"), each application of
- $NP \rightarrow NP N$
attaches one more terminal on the right, but leaves **where** you attach it ambiguous.

The Combinatorial Problem

- Parsing “ $w_0 w_1 \dots w_k$ ” under that single binary-branching rule is exactly the problem of **parenthesizing** (or building a full binary tree over) $k + 1$ leaves.
- Each parse corresponds to one way of inserting parentheses in a product of $k + 1$ factors:

$$w_0 \cdot w_1 \cdot \dots \cdot w_k$$

There is no crossing or reordering—just different ways to group them.

Catalan Numbers Count Binary Trees

The number of ways to fully parenthesize $n + 1$ items (or equivalently, the number of distinct full binary trees with n internal nodes) is the n -th **Catalan number** $\text{Cat}(n)$.

Closed-form:

$$\text{Cat}(n) = \frac{1}{n+1} \binom{2n}{n}$$

So for n recursions, you get $\text{Cat}(n)$ distinct parses.

Parsing Natural Language

k (recursions)	leaves = k+1	parses = C_k
0	1	$C_0 = 1$
1	2	$C_1 = 1$
2	3	$C_2 = 2$
3	4	$C_3 = 5$
4	5	$C_4 = 14$
5	6	$C_5 = 42$
6	7	$C_6 = 132$

“natural language processing book” uses $k = 3$ applications of $NP \rightarrow NP N \rightarrow$ **5** parses ($C_3=5$).

– Adding yet another word ($k=4$) jumps to 14 parses, and so on.

Parsing Natural Language

Compact Representation: Parse Forests

- ❑ Even though there may be exponentially many full trees, modern CFG parsers (e.g. CYK) build a **packed parse forest** in $O(n^3)$ time (worst case) and $O(n^2)$ space.
- ❑ This forest encodes all parses without enumerating them one-by-one.

The Second Bottleneck: Disambiguation

❑ Rule Acquisition

Which recursive rules (NP coordination, PP attachment, etc.) are allowed?

Preference Acquisition

Which of the many valid parses is most plausible for a given sentence?

Relies on lexical, semantic and contextual cues.

Treebanks as a Unified Solution

- ❑ A **treebank** is a large corpus of sentences manually annotated with their “correct” parse trees.
 - **Addresses both bottlenecks:**
- ❑ Explicitly records which grammatical constructions actually occur.
- ❑ Provides data to learn statistical or neural models that score/rank competing parses.
 - **Example:** Penn Treebank tags coordination and PP-attachments in Wall Street Journal text; a parser trained on it learns both the rules and their preferences.

Parsing Natural Language

- ❑ Parsers use the CFG (or dependency rules) to build a parse forest in polynomial time.
- ❑ A **statistical/neural disambiguation model**, trained on treebank data, then selects the single best parse (or k-best) for downstream tasks.
- ❑ In this way, treebanks overcome the **combinatorial explosion** of possible structures, and the **knowledge-acquisition** challenge of deciding which structure is correct in context.

Shallow Parsing

A parser takes input in the form of sequence of tokens and produces output in the form of parse tree.

•Key Elements in Shallow Parsing:

- **Noun Phrase (NP):** A chunk that contains a noun and its modifiers.
 - Example: "**The red car**" in the sentence "The red car is fast."
- **Verb Phrase (VP):** A chunk that contains a verb and any objects or modifiers.
 - Example: "**is driving fast**" in the sentence "He is driving fast."
- **Prepositional Phrase (PP):** A chunk that starts with a preposition and includes its object.
 - Example: "**in the park**" in the sentence "She sat in the park."

Shallow Parsing

- Shallow parsing typically proceeds through these steps:
 - **Tokenization**: Breaking the text into individual words or tokens.
 - **POS Tagging**: Assigning part-of-speech tags (e.g., noun, verb, adjective) to each token.
 - **Chunking**: Grouping the tokens into larger units (phrases) based on syntactic patterns and rules.
- **Example of Shallow Parsing**:
 - For the sentence "**The cat sat on the mat**," shallow parsing would break it down into chunks like this:
 - **[NP The cat] [VP sat] [PP on [NP the mat]]**

Shallow Parsing



Learning from Experience, Sharing with Passion

•Shallow Parsing vs. Full Parsing:

- **Shallow Parsing:** Focuses on identifying higher-level structures (phrases) without going deeper into hierarchical relations between words. It's faster and often used for information extraction, named entity recognition, and simple syntactic analysis.
- **Full Parsing (Deep Parsing):** Constructs a complete parse tree representing the full syntactic structure of the sentence, capturing relationships between all parts of the sentence. It's more computationally intensive and used for deeper linguistic analysis.

•Applications of Shallow Parsing:

- **Information Extraction:** Identifying key pieces of information (e.g., names, locations) in text.
- **Named Entity Recognition (NER):** Finding and classifying entities (e.g., people, organizations) within a sentence.
- **Text Summarization:** Simplifying text by extracting relevant chunks.

Shallow Parsing

Modeling a Sentence using Phase Structure

Every sentence consists of an internal structure which could be modeled with the phrase structure.

Algorithm : Steps

- ‡ Apply rules on an proposition
- ‡ The base proposition would be :
 S (the root, ie the sentence).
- ‡ The first production rule would be :
 (NP = noun phrase, VP = verb phrase)
 S -> (NP, VP)
- ‡ Apply rules for the 'branches'
 NP -> noun **VP -> verb, NP**
- ‡ The verb and noun have terminal nodes which could be any word in the lexicon for the appropriate category.
- ‡ The end is a tree with the words as terminal nodes, which is referred as the sentence.

Shallow Parsing

Statement: He ate the pizza

$G = (V, T, P, S)$

V- set of variables = { S, NP, VP, DT, N, V, PRO }

T- Set of terminals = { he, ate, the, Pizza }

P- Production Rule

S- start sym.

P: { $S \rightarrow NP \ VP$

$NP \rightarrow PRO$

$NP \rightarrow ART \ N$

$VP \rightarrow V \ NP$

$PRO \rightarrow He$

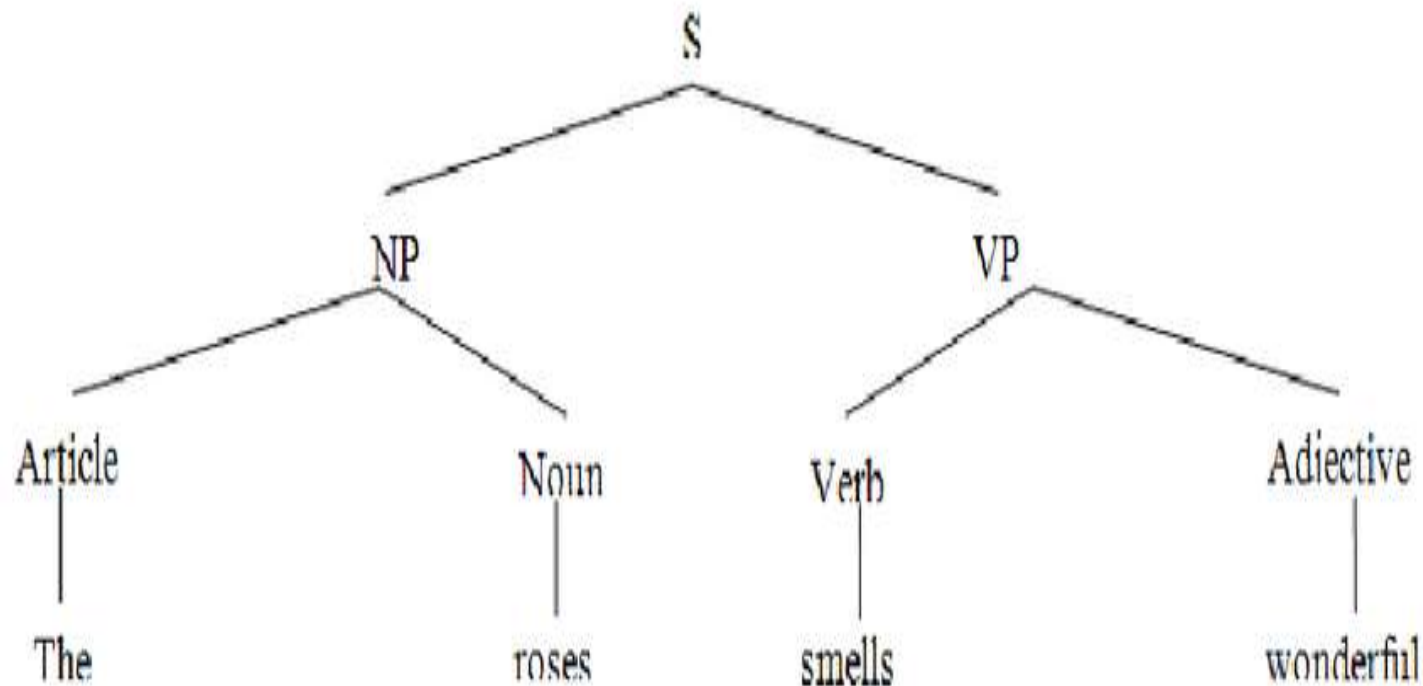
$DT \rightarrow the$

$V \rightarrow ate$

$N \rightarrow Pizza \}$

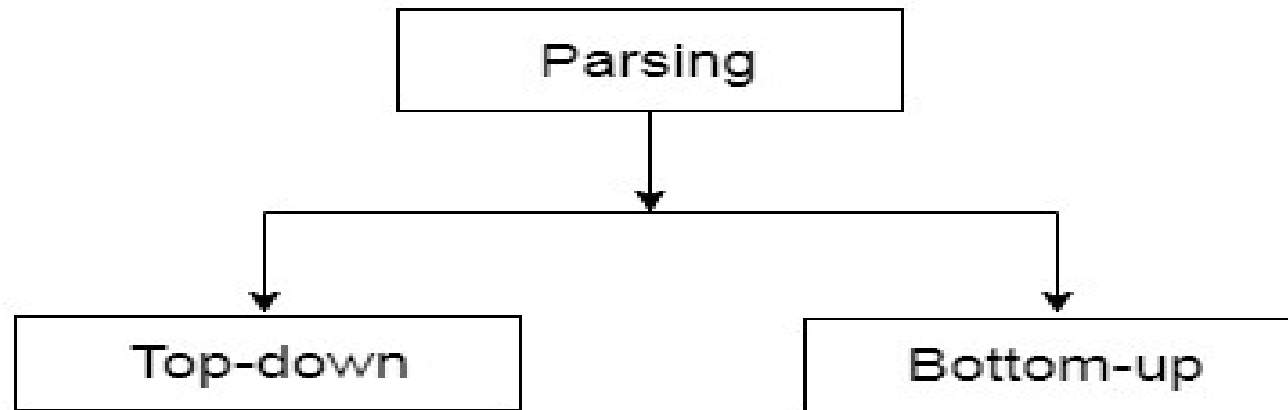
Shallow Parsing

Parse tree for the sentence “The rose smells wonderful”



Shallow Parsing

Parsing is of two types: top down parsing and bottom up parsing.

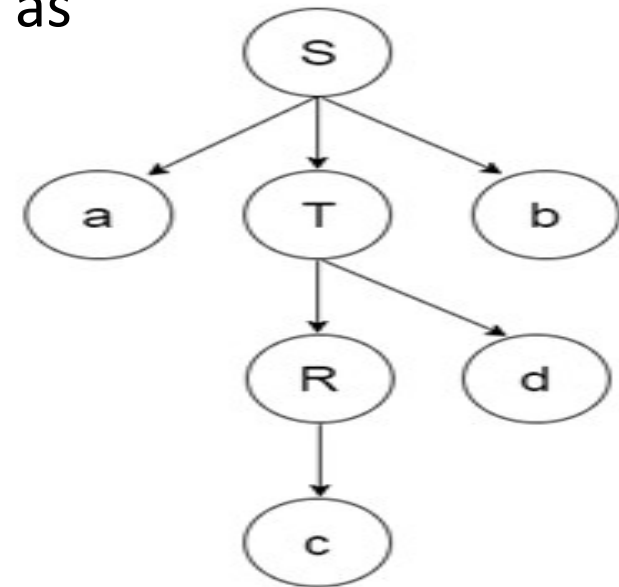


Shallow Parsing

Top down parsing

- ❖ The top down parsing is known as recursive parsing or predictive parsing.
- ❖ In the top down parsing, the parsing starts from the start symbol and transform it into the input symbol.

Parse Tree representation of input string "acdb" is as follows:



Shallow Parsing

Bottom up parsing

- ❖ Bottom up parsing is also known as shift-reduce parsing.
- ❖ Bottom up parsing is used to construct a parse tree for an input string.
- ❖ In the bottom up parsing, the parsing starts with the input symbol and construct the parse tree up to the start symbol by tracing out the rightmost derivations of string in reverse.

Parse Tree representation of input string "id * id" is as follows:

$E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow id$
 $F \rightarrow T$
 $F \rightarrow id$

id * id

Step 1

F * id
|
id

Step 2

T * id
|
F
|
id

Step 3

T * id
|
F
|
id

Step 4

T
/ | \
T * F
/ \
F id
|
id

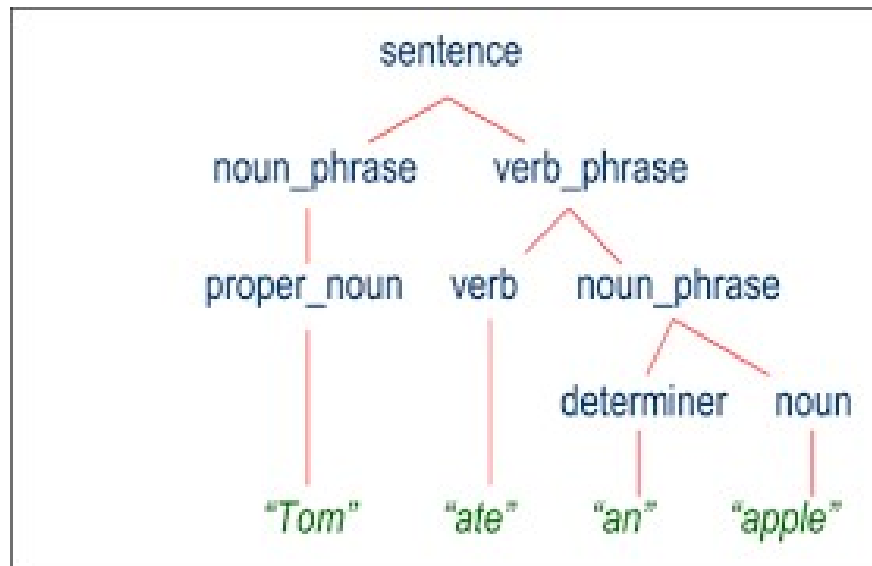
Step 5

E
|
T
/ | \
T * F
/ \
F id
|
id

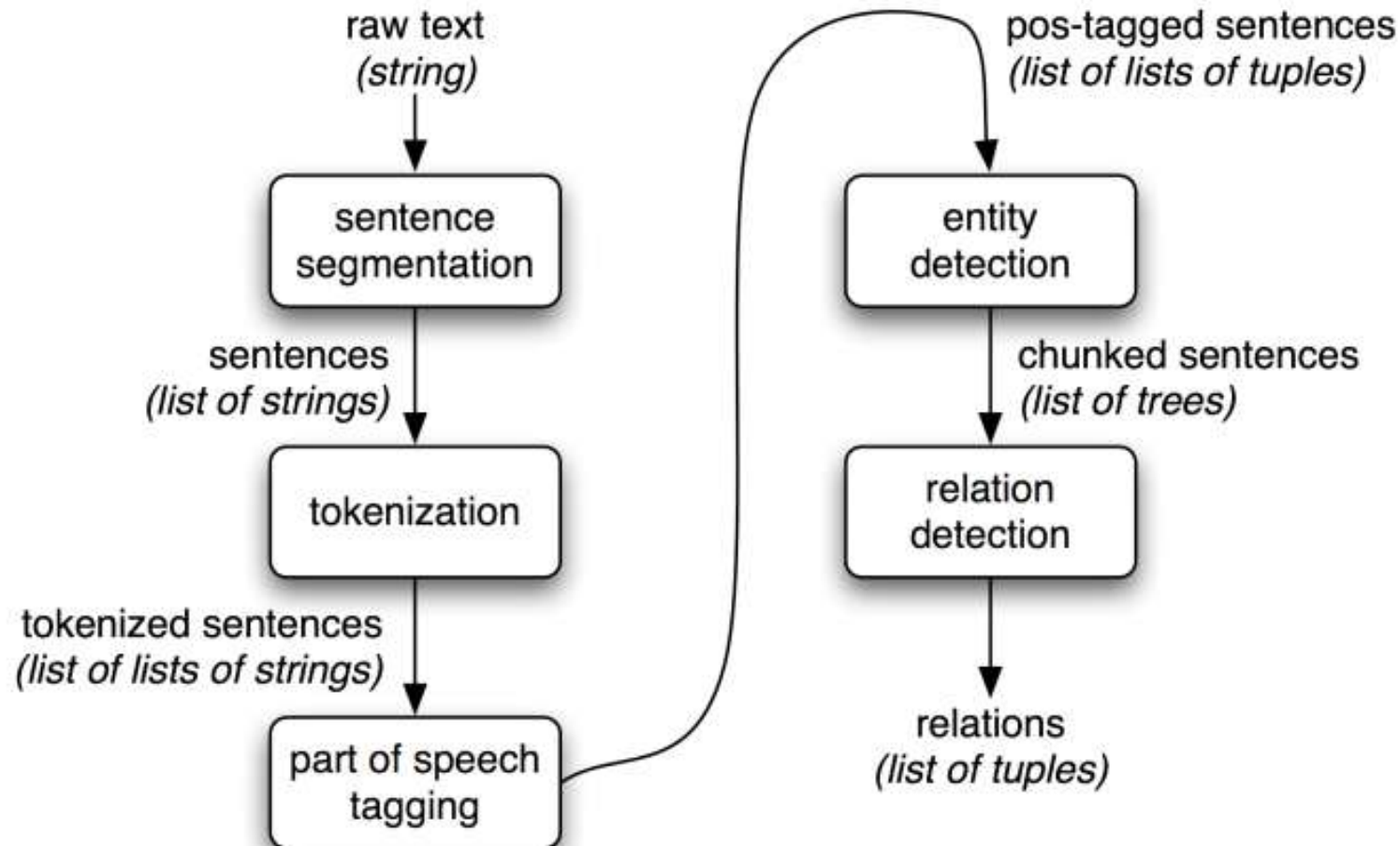
Step 6

Shallow Parsing

❖ Shallow parsing (also chunking or light parsing) is an analysis of a sentence which first identifies constituent parts of sentences (nouns, verbs, adjectives, etc.) and then links them to higher order units that have discrete grammatical meanings (noun groups or phrases, verb groups, etc.).



Shallow Parsing



Shallow Parsing

```
import nltk
sentence = [
    ("a", "DT"),
    ("clever", "JJ"),
    ("fox", "NN"),
    ("was", "VBP"),
    ("jumping", "VBP"),
    ("over", "IN"),
    ("the", "DT"),
    ("wall", "NN")
]
grammar = "NP:{<DT>?<JJ>*<NN>}"
Reg_parser = nltk.RegexpParser(grammar)
Reg_parser.parse(sentence)
Output = Reg_parser.parse(sentence)
Output.draw()
```

Dependency-based Parsing

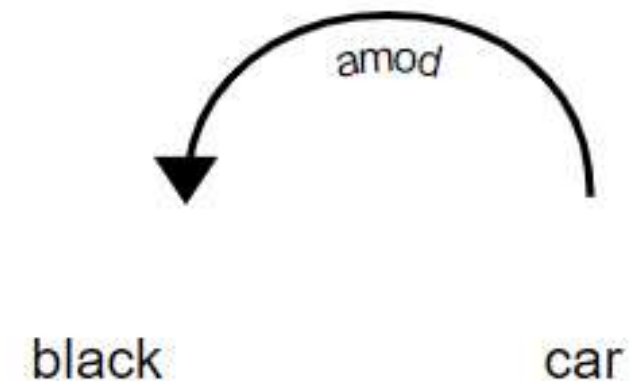
- ❖ **Dependency parsing** displays only relationships between words and their constituents while **constituency parsing** displays the entire sentence structure and relationships.
- ❖ All the nodes in the tree are words; links among words are labeled by syntactic function.
- ❖ A dependency parse tree is a graph where the set of vertices contains the words in the sentence, and each edge in connects two words.
- ❖ The graph must satisfy three conditions:
 - There has to be a single root node with no incoming edges.
 - For each node in , there must be a path from the root to .
 - Each node except the root must have exactly 1 incoming edge.
 - Additionally, each edge in has a type, which defines the grammatical relation that occurs between the two words.

Dependency-based Parsing

Each relationship:

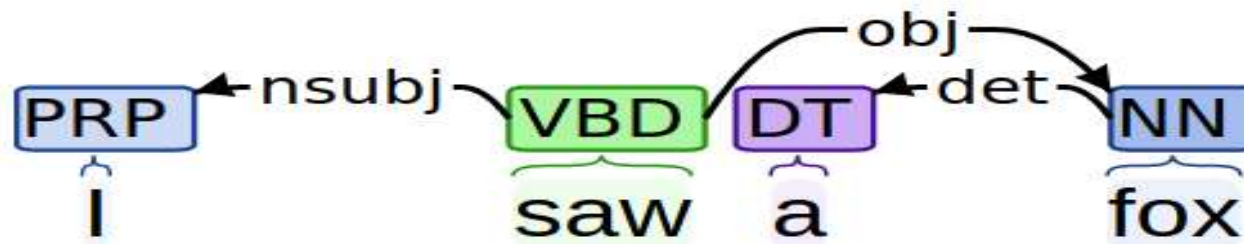
- ❖ Has one **head** and a **dependent** that modifies the **head**.
- ❖ Is labeled according to the nature of the dependency between the **head** and the **dependent**. These labels can be found at [Universal Dependency Relations](#).

A relationship between ***car*** and ***black*** because *black* modifies the meaning of *car*. Here, *car* acts as the **head** and *black* is a **dependent** of the head. The nature of the relationship here is **amod** which stands for “**Adjectival Modifier**”. It is an adjective or an adjective phrase that modifies a noun.



Dependency-based Parsing

Dependency-based Parsing



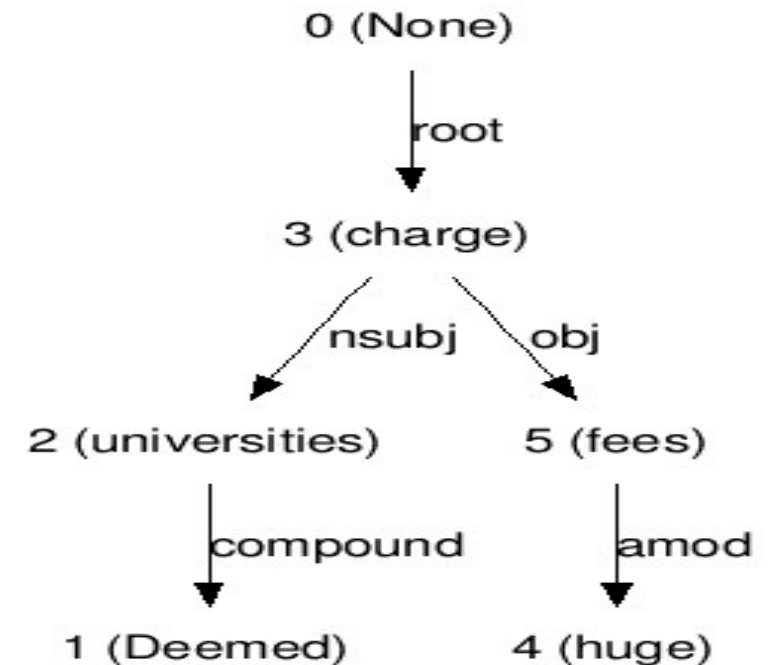
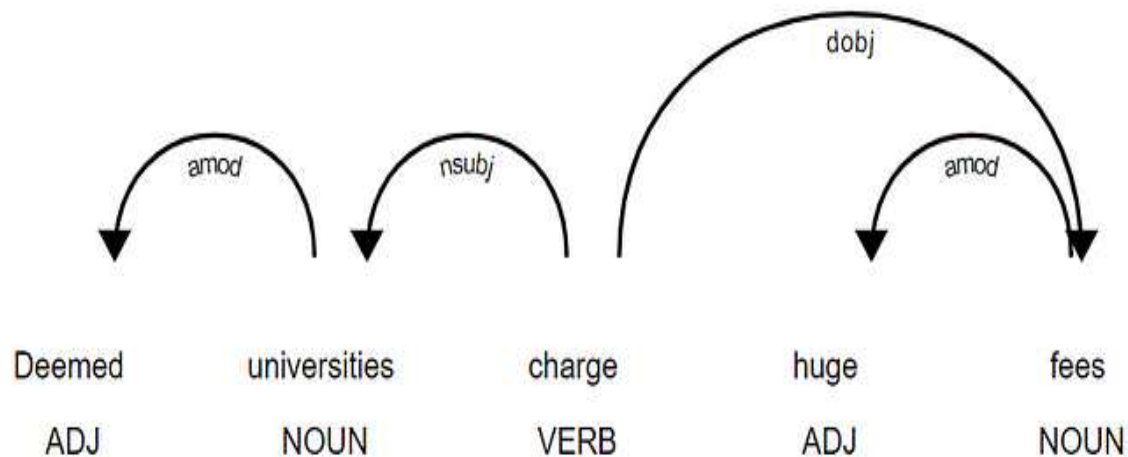
The root of the tree is the **verb** of the sentence, and edges between words describe their relationships.

For example, the word "saw" has an outgoing edge of type *nsubj* to the word "I", meaning that "I" is the nominal subject of the verb "saw".

In this case, we say that "I" *depends on* "saw".

Representation of Syntactic Structure

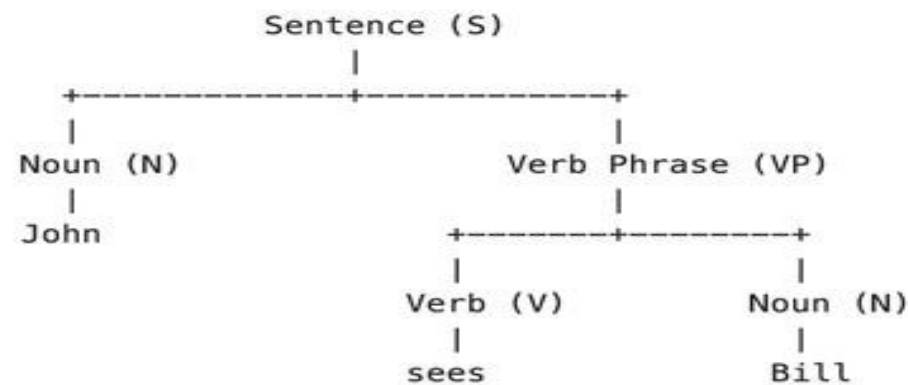
Dependency-based Parsing



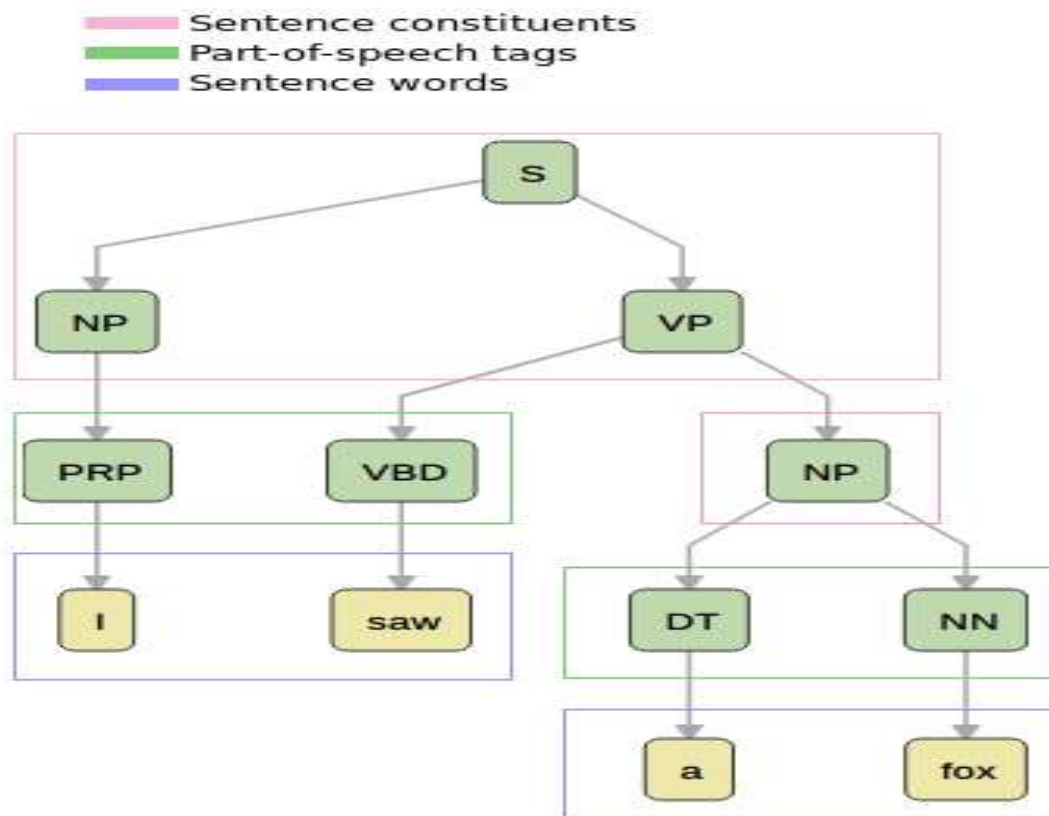
Phrase Structured Parsing

Constituency -based Parsing

- ❖ Constituency Parsing is the process of analysing the sentences by breaking down it into sub-phrases also known as constituents.
- ❖ These sub-phrases belong to a specific category of grammar like NP (noun phrase) and VP(verb phrase).
- ❖ Constituency parsing aims to extract a constituency-based parse tree from a sentence that represents its syntactic structure according to a [phrase structure grammar](#) or CFG



Phrase Structured Parsing

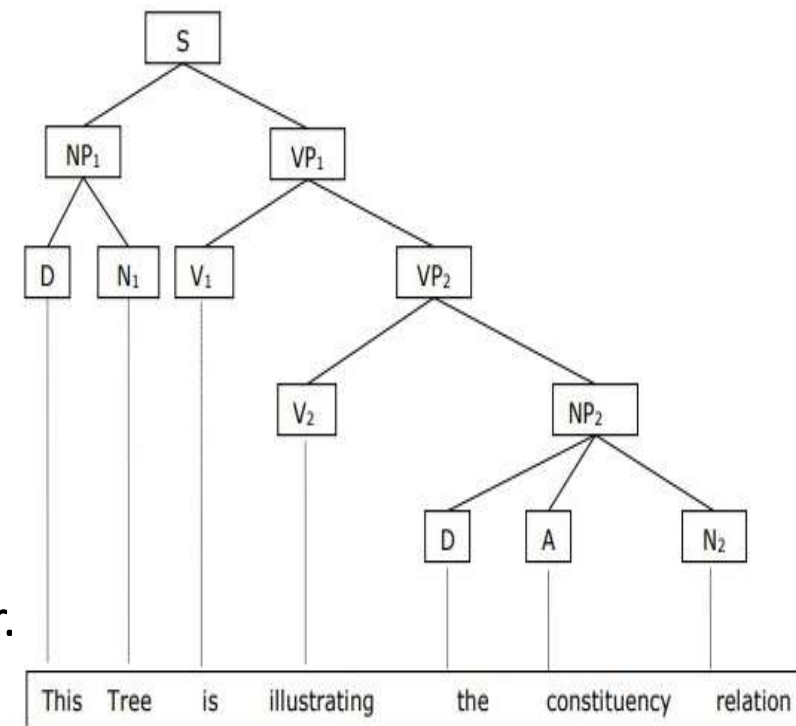


Parsing Natural Language

Constituency Grammar (CG)

- ❖ It is also known as Phrase structure grammar. It is called constituency Grammar as it is based on the constituency relation. It is the opposite of dependency grammar.
- ❖ Before deep dive into the discussion of CG, let's see some fundamental points about constituency grammar and constituency relation.
- ❖ All the related frameworks view the sentence structure in terms of constituency relation.
- ❖ To derive the constituency relation, we take the help of subject-predicate division of Latin as well as Greek grammar.
- ❖ Here we study the clause structure in terms of noun phrase NP and verb phrase VP.

Sentence: This tree is illustrating the constituency relation



Parsing Natural Language

Dependency Grammar (DG)

❖ It is opposite to the constituency grammar and is based on the dependency relation. **Dependency grammar** (DG) is opposite to constituency grammar because it lacks phrasal nodes.

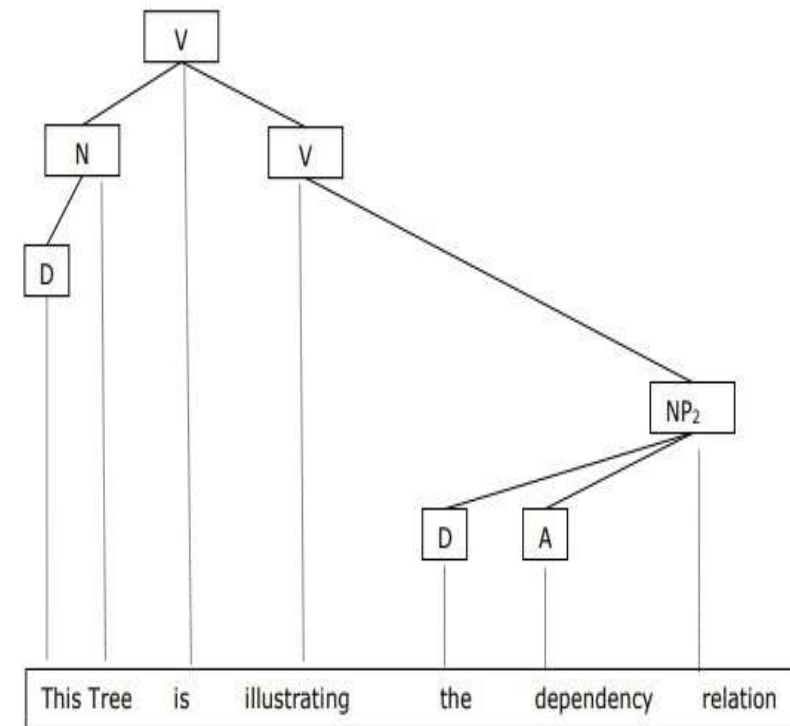
❖ Before deep dive into the discussion of DG, let's see some fundamental points about Dependency grammar and Dependency relation.

❖ In Dependency Grammar, the words are connected to each other by directed links.

❖ The verb is considered the center of the clause structure.

❖ Every other syntactic unit is connected to the verb in terms of directed link. These syntactic units are called dependencies.

Sentence: This tree is illustrating the dependency relation



Parsing Natural Language

Constituency Parsing	Dependency Parsing
Constituency parsing focuses on identifying the constituent structure of a sentence, such as noun phrases and verb phrases.	Dependency parsing focuses on identifying the grammatical relationships between words in a sentence, such as subject-verb relationships.
Constituency parsing uses phrase structure grammar, such as context-free grammar or dependency grammar.	Dependency parsing uses dependency grammar, which represents the relationships between words as labeled directed arcs.
Constituency parsing is based on a top-down approach, where the parse tree is built from the root node down to the leaves.	Dependency parsing is based on a bottom-up approach, where the parse tree is built from the leaves up to the root.
Constituency parsing represents a sentence as a tree structure with non-overlapping constituents.	Dependency parsing represents a sentence as a directed graph, where words are represented as nodes and grammatical relationships are represented as edges.

Parsing Natural Language



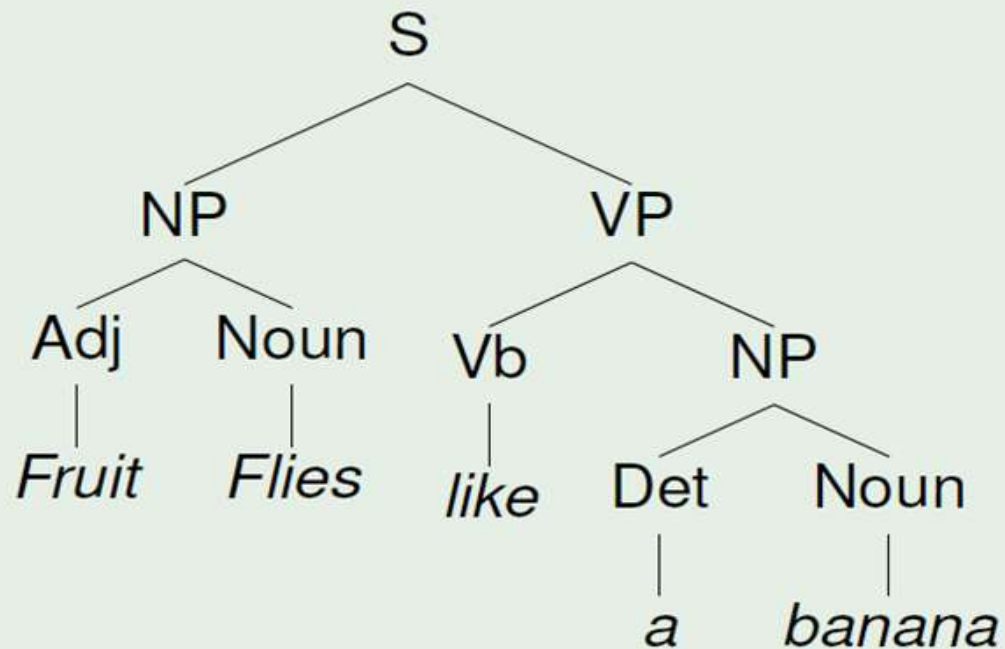
Learning from Experience, Sharing with Passion

Constituency Parsing	Dependency Parsing
Constituency parsing is more suitable for natural language understanding tasks.	Dependency parsing is more suitable for natural language generation tasks and dependency-based machine learning models.
Constituency parsing is more expressive and captures more syntactic information, but can be more complex to compute and interpret.	Dependency parsing is simpler and more efficient, but may not capture as much syntactic information as constituency parsing.
Constituency parsing is more appropriate for languages with rich morphology such as agglutinative languages.	Dependency parsing is more appropriate for languages with less morphological inflection like English and Chinese.
Constituency parsing is more suitable for natural language understanding tasks.	Dependency parsing is more suitable for natural language generation tasks and dependency-based machine learning models.
Constituency parsing is used for more traditional NLP tasks like Named Entity Recognition, Text classification.	Dependency parsing is used for more advanced NLP tasks like Machine Translation, Language Modeling, and

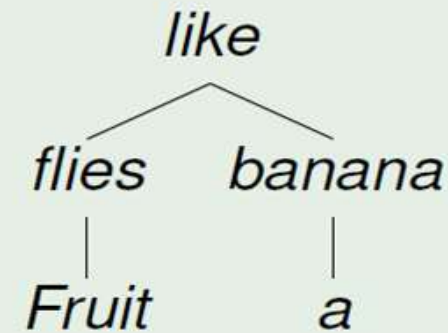
Parsing Natural Language

Fruit flies like a banana

Constituency Structure



Dependency Structure





Lecture-#8

Natural Language Processing

A8706

Syntax Analysis

Treebanks: A Data-Driven Approach to Syntax,



Learning from Experience, Sharing with Passion

- Parsing recovers information that is not explicit in the input sentence.
- This implies that a parser requires some **knowledge(syntactic rules)** in addition to the input sentence about the kind of syntactic analysis that should be produced as output.
- **One method to provide such knowledge** to the parser is to write down a grammar of the language – **a set of rules** of syntactic analysis **as a CFGs**.
- In natural language, it is far **too complex** to simply **list all the syntactic rules** in terms of a CFG.
- The second knowledge acquisition problem- **not only do we need to know the syntactic rules** for a particular language, but we also need to know which **analysis is the most plausible**(probably) for a given input sentence.
- The **construction of treebank is a data driven approach to syntax analysis** that allows us to address both of these knowledge acquisition bottlenecks in one stroke.
- A treebank is simply a **collection of sentences** (also called a corpus of text), where each sentence is provided a **complete syntax analysis**.
- The syntactic analysis for each sentence has been judged by a **human expert** as the most possible analysis for that sentence.

Treebanks: A Data-Driven Approach to Syntax,



Learning from Experience, Sharing with Passion

- ❖ A lot of care is taken during the **human annotation process** to ensure that a consistent treatment is provided across the treebank for related grammatical phenomena.
- ❖ There is no **set of syntactic rules** or **linguistic grammar** explicitly provided by a treebank, and typically there is no list of syntactic constructions provided explicitly in a treebank.
- ❖ **A detailed set of assumptions about the syntax is typically used** as an **annotation** guideline to help the human experts produce the **single-most plausible syntactic analysis** for each sentence in the corpus.
- ❖ Treebanks provide a **solution to the two kinds** of knowledge acquisition **bottlenecks**.
- ❖ Treebanks solve the first knowledge acquisition problem of **finding the grammar** underlying the syntax analysis because the **syntactic analysis is directly** given instead of a grammar.
- ❖ In fact, the parser does not necessarily need any explicit grammar rules as long as it can faithfully produce a syntax analysis for an input sentence.
- ❖ Treebank solve the second knowledge acquisition problem as well.
- ❖ Because each sentence in a treebank has been given its **most plausible(probable)** syntactic analysis, **supervised machine learning methods** can be used **to learn a scoring function** over **all possible** syntax analyses.

Trebanks: A Data-Driven Approach to Syntax,



Learning from Experience, Sharing with Passion

CFG: CFG consists of a finite set of grammar rules having the following four components

- ❖ Set of Non-Terminals
- ❖ Set of Terminals
- ❖ Set of Productions
- ❖ Start Symbol

Set of Non-terminals

It is represented by V . The non-terminals are syntactic variables that denote the sets of strings, which helps in defining the language that is generated with the help of grammar.

Set of Terminals

It is also known as tokens and represented by Σ . Strings are formed with the help of the basic symbols of terminals.

Set of Productions

It is represented by P . The set gives an idea about how the terminals and nonterminals can be combined. Every production consists of the following components:

Non-terminals,

Arrow,

Terminals (the sequence of terminals).

The left side of production is called non-terminals while the right side of production is called terminals.

Start Symbol

The production begins from the start symbol. It is represented by symbol S . Non-terminal symbols are always designated as start symbols.

Trebanks: A Data-Driven Approach to Syntax,

Left Most Derivation (LMD) and Derivation Tree :

- ❖ Leftmost derivation of a string from starting symbol S is done by replacing leftmost non-terminal symbol by RHS of corresponding production rule.
- ❖ For example, the leftmost derivation of string $abab$ from grammar G above is done as : $\underline{S} \Rightarrow a\underline{S}b \Rightarrow ab\underline{S}ab \Rightarrow abab$
- ❖ The symbols underlined are replaced using production rules.

Right Most Derivation (RMD) :

- ❖ Rightmost derivation of a string from starting symbol S is done by replacing rightmost non-terminal symbol by RHS of corresponding production rule.
 - ❖ e.g.; The rightmost derivation of string $abab$ from grammar G above is done as : $\underline{S} \Rightarrow S\underline{S} \Rightarrow Sa\underline{S}b \Rightarrow \underline{S}ab \Rightarrow a\underline{S}bab \Rightarrow abab$
 - ❖ The symbols underlined are replaced using production rules.
- The derivation tree for $abab$ using rightmost derivation

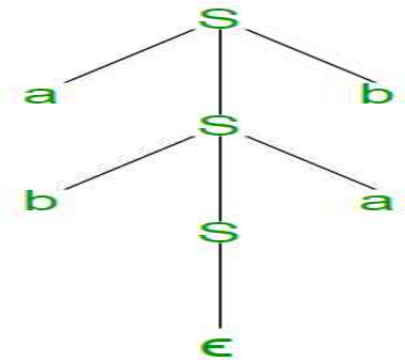


Figure 1

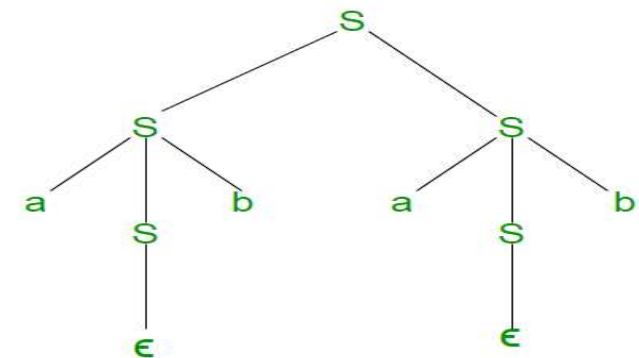


Figure 2

Ambiguous Context Free Grammar :

- ❖ A context free grammar is called ambiguous if there exists more than one LMD or more than one RMD for a string which is generated by grammar.
- ❖ There will also be more than one derivation tree for a string in ambiguous grammar.
- ❖ The grammar described above is ambiguous because there are two derivation trees (Figure 1 and Figure 2).
- ❖ There can be more than one RMD for string abab which are:
 $\underline{S} \Rightarrow S\underline{S} \Rightarrow Sa\underline{S}b \Rightarrow \underline{S}ab \Rightarrow a\underline{S}bab \Rightarrow abab$
 $\underline{S} \Rightarrow a\underline{S}b \Rightarrow ab\underline{S}ab \Rightarrow abab$

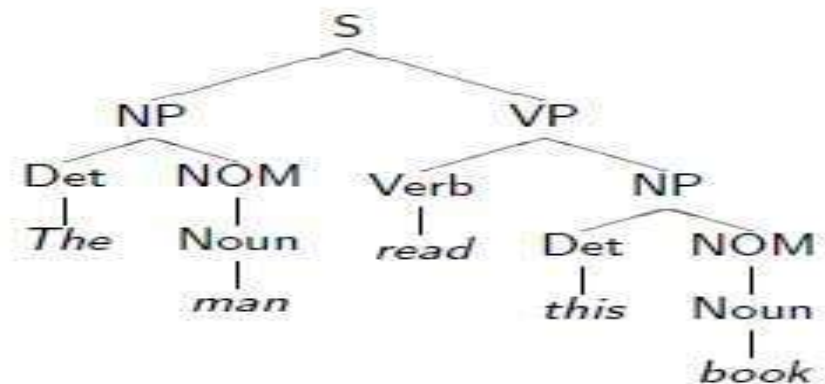
Treebanks: A Data-Driven Approach to Syntax,

Application of grammar rewrite rules

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

$S \rightarrow NP VP$
 $\rightarrow Det NOM VP$
 $\rightarrow The NOM VP$
 $\rightarrow The Noun VP$
 $\rightarrow The man VP$
 $\rightarrow The man Verb NP$
 $\rightarrow The man read NP$
 $\rightarrow The man read Det NOM$
 $\rightarrow The man read this NOM$
 $\rightarrow The man read this Noun$
 $\rightarrow The man read this book$

Parse tree



Text Normalization

Removing Stopwords :

Stop Words: A stop word is a commonly used word that a search engine has been programmed to ignore, both when indexing entries for searching and when retrieving them as the result of a search query.

```
import nltk
from nltk.corpus import stopwords
print(stopwords.words('english'))
```

Text Normalization

Removing Stopwords :

['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it's", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will', 'just', 'don', "don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren', "aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven', "haven't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "shouldn't", 'wasn', "wasn't", 'weren', "weren't", 'won', "won't", 'wouldn', "wouldn't"]

Total-179

Text Normalization

Removing Stopwords :

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
example_sent = """This is a sample sentence, showing off the stop words filtration."""
stop_words = set(stopwords.words('english'))
word_tokens = word_tokenize(example_sent)
# converts the words in word_tokens to lower case and then checks whether
#they are present in stop_words or not
filtered_sentence = [w for w in word_tokens if not w.lower() in stop_words]
#with no lower case conversion
filtered_sentence = []
for w in word_tokens:
    if w not in stop_words:
        filtered_sentence.append(w)
print(word_tokens)
print(filtered_sentence)
```

Text Normalization

Removing Stopwords:

['This', 'is', 'a', 'sample', 'sentence', ',', 'showing', 'off', 'the', 'stop', 'words', 'filtration', '.']

['This', 'sample', 'sentence', ',', 'showing', 'stop', 'words', 'filtration', '.']

Text Normalization

Correcting Words:

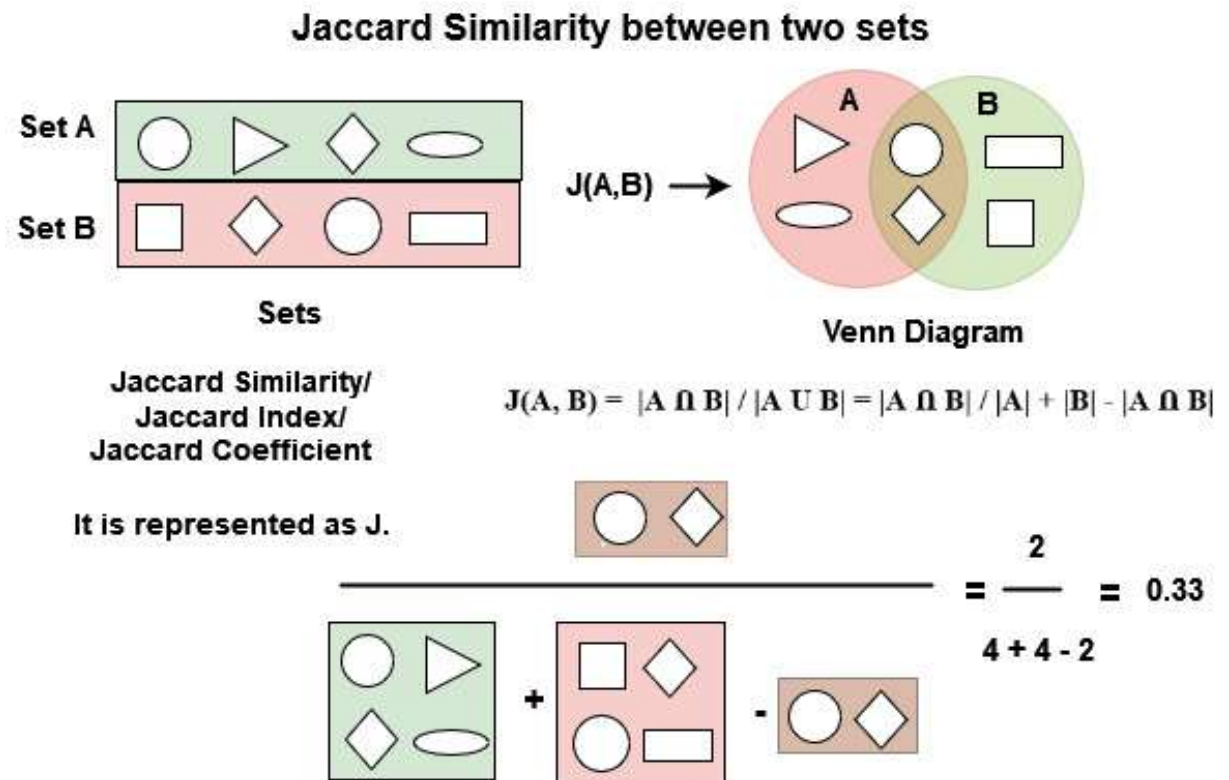
- ❖ Each method takes a list of misspelled words and gives the suggestion of the correct word for each incorrect word.
- ❖ It tries to find a word in the list of correct spellings that has the shortest distance and the same initial letter as the misspelled word.
- ❖ It then returns the word which matches the given criteria.
- ❖ The methods can be differentiated on the basis of the distance measure they use to find the closest word. 'words' package from nltk is used as the dictionary of correct words.
- ❖ **Using Jaccard distance Method:**
- ❖ It is mainly used to measure similarities between two sets or between two asymmetric binary vectors.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

- ❖ **Jaccard distance=1- J(A, B)** where J(A, B) is Jaccard index

Text Normalization

Correcting Words:



Jaccard distance = $1 - J(A, B) = 1 - 0.33 = 0.66$

Text Normalization

Correcting Words:

```
# importing the nltk suite
import nltk
# importing jaccard distance
# and ngrams from nltk.util
from nltk.metrics.distance import jaccard_distance
from nltk.util import ngrams
# Downloading and importing
# package 'words' from nltk corpus
nltk.download('words')
from nltk.corpus import words
correct_words = words.words()
w1 = set('cat')
w2 = set('cap')
nltk.jaccard_distance(w1, w2)
```

O/P: 0.5

Text Normalization

Correcting Words:

```
# list of incorrect spellings
# that need to be corrected
incorrect_words=['happy', 'azmaing', 'intelliengt']

# loop for finding correct spellings
# based on jaccard distance
# and printing the correct word
for word in incorrect_words:
    temp = [(jaccard_distance(set(ngrams(word, 2)),
                                set(ngrams(w, 2))),w)
             for w in correct_words if w[0]==word[0]]
    print(sorted(temp, key = lambda val:val[0])[0][1])
```

O/P:

happy

Amazing

intelligent

Text Normalization

`ngrams(word, 2):`

- This generates **bigrams** (sequences of two consecutive characters) from the given word. For example, for the word "happy", it will generate the bigrams ['ha', 'ap', 'pp', 'py'].

`set(ngrams(word, 2)):`

- The `ngrams()` function produces a list, but `set()` converts that list into a set of unique bigrams. This is useful because the Jaccard distance works on set comparisons, not lists. For example, `set(['ha', 'ap', 'pp', 'py'])` for the word "happy".

Text Normalization

•Example: Jaccard Distance Between 'MAKE' and 'CAKE'

- **Step 1: Convert words to sets of bigrams (2-grams):**
 - First, convert the words into sets of character bigrams (two consecutive characters).
- For 'MAKE':
 - Bigrams of MAKE={ 'MA', 'AK', 'KE' }
- For 'CAKE':
 - Bigrams of CAKE={ 'CA', 'AK', 'KE' }
- **Step 2: Find the intersection:**
 - The **intersection** is the set of common bigrams between the two sets.
 - $A \cap B = \{ 'AK', 'KE' \}$
- So, the size of the intersection is 2.

Text Normalization

•Step 3: Find the union:

- The **union** is the set of all unique bigrams from both sets.

$$- A \cup B = \{'MA', 'AK', 'KE', 'CA'\}$$

– So, the size of the union is 4.

– Step 4: Compute the Jaccard similarity:

$$\text{Jaccard Similarity} = \frac{|A \cap B|}{|A \cup B|} = \frac{2}{4} = 0.5$$

– Step 5: Compute the Jaccard distance:

$$\text{Jaccard Distance} = 1 - \text{Jaccard Similarity} = 1 - 0.5 = 0.5$$

Text Normalization

Correcting Words:

Using Edit distance Method: The distance between the source string and the target string is the minimum number of edit operations (deletions, insertions, or substitutions) required to transform the source into the target. The lower the distance, the more similar the two strings.

importing the nltk suite

import nltk

importing edit distance

from nltk.metrics.distance import edit_distance

Downloading and importing package 'words'

nltk.download('words')

from nltk.corpus import words

correct_words = words.words()

Text Normalization

Correcting Words:

```
import nltk  
w1 = 'cat'  
w2 = 'cap'  
nltk.edit_distance(w1, w2)
```

O/P:1

Input: *str1 = "cat", str2 = "cut"*

Output: *1*

Explanation: *We can convert str1 into str2 by replacing 'a' with 'u'.*

Input: *str1 = "sunday", str2 = "saturday"*

Output: *3*

Explanation: *Last three and first characters are same. We basically need to convert "un" to "atur". This can be done using below three ope*

Text Normalization

Correcting Words:

```
# list of incorrect spellings
# that need to be corrected
incorrect_words=['happpy', 'azmaing', 'intelliengt']
# loop for finding correct spellings
# based on edit distance and
# printing the correct words
for word in incorrect_words:
    temp = [(edit_distance(word, w),w) for w in correct_words if w[0]==word[0]]
    print(sorted(temp, key = lambda val:val[0])[0][1])
```

O/P:

happy

aiming

intelligent

Text Normalization

The **edit distance** (or Levenshtein distance) method is useful in various Natural Language Processing (NLP) and text processing tasks. Here's how it can be applied and why it is important:

1. Spelling Correction:

- **Problem:** Given a misspelled word, how do you suggest the most likely correct word?
- **How Edit Distance Helps:** It calculates the number of changes (insertions, deletions, substitutions) needed to turn the misspelled word into a correct one. The word with the smallest edit distance is likely to be the intended correct word.
- **Example:**
 - Misspelled word: "happpy"
 - Closest correct word by edit distance: "happy" (edit distance of 1)
 - This tells us only one character needs to be removed, making it likely that "happy" was the intended word.

2. Autocorrect Systems:

- **Problem:** When typing on a keyboard or mobile device, people make frequent typos.
- **How Edit Distance Helps:** Autocorrect algorithms use edit distance to suggest the correct word based on minimal changes to the typed word.
- **Example:** Typing "teh" suggests "the" because it has an edit distance of 1 (a simple character swap).

Text Normalization

3. Plagiarism Detection:

- **Problem:** Detecting if two documents are similar even if minor changes (word swaps, misspellings, etc.) are made.
- **How Edit Distance Helps:** By calculating the edit distance between sentences, paragraphs, or words in documents, systems can estimate the level of similarity. A smaller edit distance implies higher similarity.

4. DNA Sequence Analysis:

- **Problem:** Comparing DNA sequences to find similarities or mutations.
- **How Edit Distance Helps:** In bioinformatics, edit distance can be used to quantify the similarity between DNA sequences, helping identify genetic mutations.
- **Example:** Comparing two DNA sequences: "AGCTT" and "ACGTT" would have an edit distance of 1 (a single substitution).

Text Normalization

5. Search Engines & Fuzzy Matching:

- **Problem:** When searching online, users may enter queries with typos or slightly incorrect words.
- **How Edit Distance Helps:** Search engines and systems use edit distance to find results that match closely even when the query has minor errors.
- **Example:** Searching for "recepie" can yield results for "recipe" due to a small edit distance.

6. Text Similarity and Clustering:

- **Problem:** Finding how similar two strings, sentences, or documents are.
- **How Edit Distance Helps:** If the edit distance between two pieces of text is small, they are likely closely related. This is useful in clustering similar text documents or finding paraphrases.
- **Example:** Comparing "Hello World" and "Hello Wrld" gives an edit distance of 1, showing they are very similar.

Text Normalization

7. Named Entity Recognition (NER):

- **Problem:** Identifying proper nouns (like names of people, places) even when there are small spelling differences.
- **How Edit Distance Helps:** In systems that try to recognize names (or brands), slight variations (like missing or extra letters) can still be detected as the same entity if the edit distance is low.

Parts of Speech (POS) Tagging

❖ part-of-speech tagging (POS tagging or PoS tagging or POST), also called grammatical tagging or word-category disambiguation.

❖ Input: I am going.

❖ Output:

with stop word

[('I', 'PRP'), ('am', 'VBP'), ('going', 'VBG'), ('.', '.')]]

Without stop word

[('I', 'PRP'), ('going', 'VBG'), ('.', '.')]]

Where

PRP- Pronoun Personal

VBP-Verb non-3rd personal singular present

VBG-Verb Gerund/ Present Participle

Parts of Speech (POS) Tagging

Sl No.	TAG	DESCRIPTION	EXAMPLE(S)
1	CC	Coordinating Conjunction	<i>and, or</i>
2	CD	Cardinal Number	<i>five, one, 2</i>
3	DT	Determiner	<i>a, the</i>
4	EX	Existential <i>there</i>	<i>there were two cars</i>
5	FW	Foreign Word	<i>d'hoevre, mais</i>
6	IN	Preposition/ Subordinating Conjunction	<i>of, in, on, that</i>
7	JJ	Adjective	<i>quick, lazy</i>
8	JJR	Adjective, comparative	<i>quicker, lazier</i>

Parts of Speech (POS) Tagging

9	JJS	Adjective, superlative	<i>quickest, laziest</i>
10	LS	List item marker	<i>2)</i>
11	MD	Verb, modal	<i>could, should</i>
12	NN	Noun, singular or mass	<i>fox, dog</i>
13	NNS	Noun, plural	<i>foxes, dogs</i>
14	NNP	Noun, proper singular	<i>John, Alice</i>
15	NNPS	Noun, proper plural	<i>Vikings, Indians, Germans</i>
16	PDT	Predeterminer	<i>both the cats</i>
17	POS	Possessive ending	<i>boss's</i>
18	PRP	Pronoun, personal	<i>me, you</i>

Parts of Speech (POS) Tagging

19	PRP\$	Pronoun, possessive	<i>our, my, your</i>
20	RB	Adverb	<i>naturally, extremely, hardly</i>
21	RBR	Adverb, comparative	<i>better</i>
22	RBS	Adverb, superlative	<i>best</i>
23	RP	Adverb, particle	<i>about, up</i>
24	SYM	Symbol	<i>%, \$</i>
25	TO	Infinitival to	<i>how to, what to do</i>
26	UH	Interjection	<i>oh, gosh, wow</i>

Parts of Speech (POS) Tagging

SI No.	TAG	DESCRIPTION	EXAMPLE(S)
27	VB	Verb, base form	<i>run, give</i>
28	VBD	Verb, past tense	<i>ran, gave</i>
29	VBG	Verb, gerund/ present participle	<i>running, giving</i>
30	VBN	Verb, past participle	<i>given</i>
31	VBP	Verb, non-3rd person singular present	<i>I think, I take</i>
32	VBZ	Verb, 3rd person singular present	<i>he thinks, he takes</i>
33	WDT	Wh-determiner	<i>which, whatever</i>
34	WP	Wh-pronoun, personal	<i>who, what</i>
35	WP\$	Wh-pronoun, possessive	<i>whose</i>

Parts of Speech (POS) Tagging

36	WRB	Wh-adverb	<i>where, when</i>
37	NP	Noun Phrase	<i>the brown fox</i>
38	PP	Prepositional Phrase	<i>in between, over the dog</i>
39	VP	Verb Phrase	<i>was jumping</i>
40	ADJP	Adjective Phrase	<i>warm and snug</i>
41	ADVP	Adverb Phrase	<i>also</i>
42	SBAR	Subordinating Conjunction	<i>whether or not</i>
43	PRT	Particle	<i>up</i>
44	INTJ	Interjection	<i>hello</i>
45	PNP	Prepositional Noun Phrase	<i>over the dog, as of today</i>
46	-SBJ	Sentence Subject	<i>the fox jumped over the dog</i>
47	-OBJ	Sentence Object	<i>the fox jumped over the dog</i>

Parts of Speech (POS) Tagging

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = " Everything is all about money."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
    # Word tokenizers is used to find the words
    # and punctuation in a string
    wordsList = nltk.word_tokenize(i)
# removing stop words from wordList
wordsList = [w for w in wordsList if not w in stop_words]
# Using a Tagger. Which is part-of-speech
# tagger or POS-tagger.
tagged = nltk.pos_tag(wordsList)
print(tagged)
```

O/P:

```
[('Everything', 'VBG'),
('money', 'NN'),
('.', '.')]

```

Parts of Speech (POS) Tagging

with stop words

```
import nltk
# download required nltk packages
# required for tokenization
nltk.download('punkt')
# required for parts of speech tagging
nltk.download('averaged_perceptron_tagger')

# input text
sentence = """"Today morning, I am Very Happy.""""

# tokene into words
tokens = nltk.word_tokenize(sentence)

# parts of speech tagging
tagged = nltk.pos_tag(tokens)

# print tagged tokens
print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','),
('I', 'PRP'),
('am', 'VBP'),
('Very', 'RB'),
('Happy', 'JJ'), (',', ',')]
```

Parts of Speech (POS) Tagging

```
# Without stop words
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = "Today morning, I am Very Happy."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
# Word tokenizers is used to find the words
# and punctuation in a string
    wordsList = nltk.word_tokenize(i)
    # removing stop words from wordList
    wordsList = [w for w in wordsList if not w in stop_words]
    # Using a Tagger. Which is part-of-speech
    # tagger or POS-tagger.
    tagged = nltk.pos_tag(wordsList)
    print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','), ('I', 'PRP'),
('Very', 'RB'),
('Happy', 'JJ'), (':', ':')]
```



Lecture-#9

Natural Language Processing

A8706

Syntax Analysis

Parsing Algorithms

Shift Reduce Parsing

- ❖ To build a parser, we need an algorithm that can perform the steps in the above rightmost derivation for any grammar and for any input string.
- ❖ Every CFG turns out to have an automata that is equivalent to it, called pushdown automata (just like regular expression can be converted to finite-state automata).
- ❖ An algorithm for parsing that is general for any given CFG and input string.
- ❖ The algorithm is called shift-reduce parsing which uses two data structures: a buffer for input symbols and a stack for storing CFG symbols.

Parsing Algorithms

This parser requires some data structures i.e.

An input buffer for storing the input string.

A stack for storing and accessing the production rules.

Basic Operations –

Shift: This involves moving symbols from the input buffer onto the stack.

Reduce: If the handle appears on top of the stack then, its reduction by using appropriate production rule is done i.e. RHS of a production rule is popped out of a stack and LHS of a production rule is pushed onto the stack.

Accept: If only the start symbol is present in the stack and the input buffer is empty then, the parsing action is called accept. When accepted action is obtained, it means successful parsing is done.

Error: This is the situation in which the parser can neither perform shift action nor reduce action and not even accept action.

Parsing Algorithms

Grammar:

$S \rightarrow S+S$

$S \rightarrow S-S$

$S \rightarrow (S)$

$S \rightarrow a$

Input string:

$a1-(a2+a3)$

Parsing Algorithms

Stack contents	Input string	Actions
\$	a1-(a2+a3)\$	shift a1
\$a1	-(a2+a3)\$	reduce by $S \rightarrow a$
\$S	-(a2+a3)\$	shift -
\$S-	(a2+a3)\$	shift (
\$S-(	a2+a3)\$	shift a2
\$S-(a2	+a3)\$	reduce by $S \rightarrow a$
\$S-(S	+a3) \$	shift +
\$S-(S+	a3) \$	shift a3
\$S-(S+a3	) \$	reduce by $S \rightarrow a$
\$S-(S+S	) \$	shift)
\$S-(S+S)	\$	reduce by $S \rightarrow S+S$
\$S-(S)	\$	reduce by $S \rightarrow (S)$
\$S-S	\$	reduce by $S \rightarrow S-S$
\$S	\$	Accept

Parsing Algorithms

Advantages:

- ❖ Shift-reduce parsing is efficient and can handle a wide range of context-free grammars.
- ❖ It can parse a large variety of programming languages and is widely used in practice.
- ❖ It is capable of handling both left- and right-recursive grammars, which can be important in parsing certain programming languages.
- ❖ The parse table generated for shift-reduce parsing is typically small, which makes the parser efficient in terms of memory usage

Parsing Algorithms

Disadvantages:

- ❖ Shift-reduce parsing has a limited lookahead, which means that it may miss some syntax errors that require a larger lookahead.
- ❖ It may also generate false-positive shift-reduce conflicts, which can require additional manual intervention to resolve.
- ❖ Shift-reduce parsers may have difficulty in parsing ambiguous grammars, where there are multiple possible parse trees for a given input sequence.
- ❖ In some cases, the parse tree generated by shift-reduce parsing may be more complex than other parsing techniques.

Parsing Algorithms

Hypergraphs and Chart Parsing:

- ❖ A chart parser is a type of parser suitable for ambiguous grammars (including grammars of natural languages).
- ❖ It uses the dynamic programming approach—partial hypothesized results are stored in a structure called a chart and can be re-used.
- ❖ A hypergraph is a generalization of a graph in which an edge can join any number of vertices.
- ❖ **Example: CYK Algorithm**

Parsing Algorithms

CYK algorithm :

- ❖ The **CYK algorithm** is a parsing algorithm for context-free grammar. It is used to check if a particular string can be derived from the language generated by a given grammar.
 - ❖ It is also called the membership algorithm as it tells whether the given string is a member of the given grammar or not.
 - ❖ It was independently developed by three Russian scientists named **Cocke, Younger, and Kasami**, hence the name CYK!
 - ❖ In a CYK algorithm, the structure of grammar should be in Chomsky normal form.
 - ❖ In addition, the CYK algorithm uses the dynamic programming or table filling algorithm.
 - ❖ The grammar will be in CNF if each rule has one of the following forms:
 - $A \rightarrow BC$ (at most two variables on the right-hand side)
 - $A \rightarrow a$ (a single terminal on the right-hand side)
 - $S \rightarrow \emptyset$ (null string)
- If the given Grammar is not in the CNF form, convert it to the CNF form before applying the CYK Algorithm.

Parsing Algorithms

CYK algorithm :

Let w be the n length string to be parsed. And G represent the set of rules in our grammar with start state S .

1. Construct a table DP for size $n \times n$.
2. If $w = \epsilon$ (empty string) and $S \rightarrow \epsilon$ is a rule in G then we accept the string else we reject.
3. For $i = 1$ to n :
 For each variable A :
 We check if $A \rightarrow b$ is a rule and $b = w_i$ for some i :
 If so, we place A in cell (i, i) of our table.
4. For $l = 2$ to n :
 For $i = 1$ to $n-l+1$:
 $j = i+l-1$
 For $k = i$ to $j-1$:
 For each rule $A \rightarrow BC$: We check if (i, k) cell contains B and $(k + 1, j)$ cell contains C :
 If so, we put A in cell (i, j) of our table.
5. We check if S is in $(1, n)$:
 If so, we accept the string
 Else, we reject.

Parsing Algorithms

CYK algorithm :

To decide the membership of any given string, we construct a triangular table where-

- ❖ Each row of the table corresponds to one particular length of sub strings.
- ❖ Bottom most row corresponds to strings of length-1.
- ❖ Second row from bottom corresponds to strings of length-2.
- ❖ Third row from bottom corresponds to strings of length-3.
- ❖ Top most row from bottom corresponds to the given string of length-n.

Parsing Algorithms

CYK algorithm :

Let our grammar G be:

$S \rightarrow AB \mid BC$

$A \rightarrow BA \mid a$

$B \rightarrow CC \mid b$

$C \rightarrow AB \mid a$

We check if **baaba** is in $L(G)$:

Parsing Algorithms

CYK algorithm :

	b	a	a	b	a
b	{B}				
a		{A,C}			
a			{A,C}		
b				{B}	
a					{A,C}

	b	a	a	b	a
b	{B}	{S,A}	ϕ	ϕ	{S,A,C}
a		{A,C}	{B}	{B}	{S,A,C}
a			{A,C}	{S,C}	{B}
b				{B}	{S,A}
a					{A,C}

We observe that S is in the cell (1, 5), Hence, the string **baaba** belongs to **L(G)**.

Minimum Spanning Trees and Dependency Parsing:

- ❖ Dependency parsing can be approached as finding a Minimum Spanning Tree in a graph where the nodes represent words in a sentence, and the edges represent potential dependencies.
- ❖ Each word in the sentence is a node, and edges represent possible grammatical relations, such as subject-verb or modifier-head relationships.
- ❖ The goal is to find the MST of this graph, where the MST edges correspond to the most probable or relevant grammatical dependencies.
- ❖ Once the MST is identified, it can be used to construct the dependency parse tree for the sentence.

Minimum Spanning Tree-based : Start with a fully-connected directed graph and find a Minimum Spanning Tree. Then Subtract its score from all incoming edges. Contracts nodes if there are cycles and recursively computes MST.

Parsing Algorithms

Minimum Spanning Trees and Dependency Parsing:

- ❖ Connects all vertices with the minimum possible number of edges
- ❖ Has the maximum weight (summation of all edges scores involved) amongst all the possible spanning trees that connect all vertices.
- ❖ The weights of edges should be relative i.e if we perform any operation on the weights, the MST shouldn't be affected
- ❖ Each vertex should have only one incoming node in an MST
- ❖ Should have no cycle

Parsing Algorithms



Learning from Experience, Sharing with Passion

*For example: For '**book that flight**', we feed the model with*

book → that,

that → book,

that → flight,

flight → that,

book → flight &

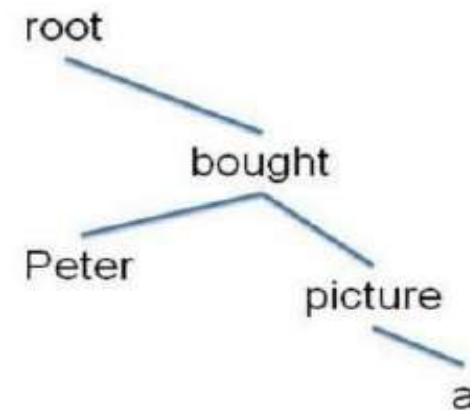
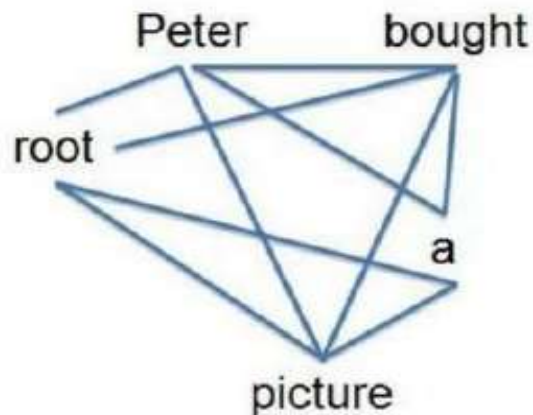
flight → book.

Their respective scores are predicted & assigned to respective edges for graph G.

<https://www.youtube.com/watch?v=JChpg5xUdL8>

Parsing Algorithms

This is equivalent to finding a spanning tree in the complete graph containing all possible arcs



Parsing Algorithms

- $$\frac{\text{\# correct dependencies}}{\text{\# of dependencies}}$$
- LAS – labeled attachment score
- UAS – unlabeled attachment score
- Which is bigger?



Lecture-#10

Natural Language Processing

A8706

Syntax Analysis

Probabilistic Context-Free Grammars:

1. Define Probabilistic context-free grammar?

A probabilistic context-free grammar is the same as context-free grammar, but here we have probabilities assigned to each rule in the grammar.

2. What is the significance of the parameters in PCFG?

The parameter $q(\alpha \rightarrow \beta)$ can be interpreted as the conditional probability of choosing rule $\alpha \rightarrow \beta$ in a left-most derivation, given that the nonterminal being expanded is α .

3. Can we apply the CKY algorithm to the Probabilistic Context-Free Grammar?

Yes, the Cocke-Younger-Kasami algorithm applies to a restricted type of PCFG, which is in Chomsky standard form (CNF).

Models for Ambiguity Resolution in Parsing



Learning from Experience, Sharing with Passion

Probabilistic Context-Free Grammars:

4. What is probabilistic parsing?

Probabilistic parsing uses dynamic programming to compute the most likely parse of a given sentence.

5. Why do we use the CYK algorithm?

We use the CYK algorithm to find whether the given string belongs to a language of grammar or not.

Models for Ambiguity Resolution in Parsing

Probabilistic Context-Free Grammars:

$N = \{S, NP, VP, PP, DT, Vi, Vt, NN, IN\}$

$S = S$

$\Sigma = \{\text{sleeps, saw, man, woman, dog, telescope, the, with, in}\}$

$R, q =$

S	→	NP	VP	1.0
VP	→	Vi		0.3
VP	→	Vt	NP	0.5
VP	→	VP	PP	0.2
NP	→	DT	NN	0.8
NP	→	NP	PP	0.2
PP	→	IN	NP	1.0

Vi	→	sleeps	1.0
Vt	→	saw	1.0
NN	→	man	0.1
NN	→	woman	0.1
NN	→	telescope	0.3
NN	→	dog	0.5
DT	→	the	1.0
IN	→	with	0.6
IN	→	in	0.4

Probabilistic Context-Free Grammars:

- The maximum-likelihood parameter estimates are

$$q_{ML}(\alpha \rightarrow \beta) = \frac{\text{Count}(\alpha \rightarrow \beta)}{\text{Count}(\alpha)}$$

where $\text{Count}(\alpha \rightarrow \beta)$ is the number of times that the rule $\alpha \rightarrow \beta$ is seen in the trees $t_1 \dots t_m$, and $\text{Count}(\alpha)$ is the number of times the non-terminal α is seen in the trees $t_1 \dots t_m$.

For example, if the rule $\text{VP} \rightarrow \text{Vt NP}$ is seen 105 times in our corpus, and the non-terminal VP is seen 1000 times, then

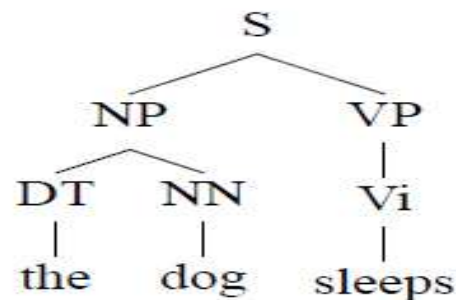
$$q(\text{VP} \rightarrow \text{Vt NP}) = \frac{105}{1000}$$

Models for Ambiguity Resolution in Parsing

Probabilistic Context-Free Grammars:

$$\begin{aligned}\sum_{\alpha \rightarrow \beta \in R: \alpha = \text{VP}} q(\alpha \rightarrow \beta) &= q(\text{VP} \rightarrow \text{Vi}) + q(\text{VP} \rightarrow \text{Vt NP}) + q(\text{VP} \rightarrow \text{VP PP}) \\ &= 0.3 + 0.5 + 0.2 \\ &= 1.0\end{aligned}$$

To calculate the probability of any parse tree t , we simply multiply together the q values for the context-free rules that it contains. For example, if our parse tree t is



then we have

$$\begin{aligned}p(t) &= q(\text{S} \rightarrow \text{NP VP}) \times q(\text{NP} \rightarrow \text{DT NN}) \times q(\text{DT} \rightarrow \text{the}) \times q(\text{NN} \rightarrow \text{dog}) \times \\ &\quad q(\text{VP} \rightarrow \text{Vi}) \times q(\text{Vi} \rightarrow \text{sleeps})\end{aligned}$$

Models for Ambiguity Resolution in Parsing



Learning from Experience, Sharing with Passion

Generative models for parsing :

- ❖ Generative models for parsing are a class of models that focus on generating the most likely parse tree for a given sentence.
- ❖ They are often probabilistic models that assign probabilities to different parse trees and aim to find the highest probability parse.

❖ Probabilistic Context-Free Grammars (PCFGs):

PCFGs are a classic generative model for parsing. They extend context-free grammars by associating probabilities with each production rule. The probability of a parse tree is calculated as the product of the probabilities of its constituent rules. Parsing involves finding the parse tree with the highest probability.

❖ Lexicalized PCFGs (L-PCFGs):

L-PCFGs enhance PCFGs by incorporating information about the words in the sentence. Instead of just relying on syntactic structure, L-PCFGs consider the words and their relationships when assigning probabilities to parse trees. This makes them more sensitive to word-specific and syntactic ambiguities.

Models for Ambiguity Resolution in Parsing



Learning from Experience, Sharing with Passion

Generative models for parsing :

❖ **Transformation-Based Models:** Transformation-based models, such as the transformation-based learning (TBL) algorithm, are generative models that iteratively transform one parse tree into another. These models learn a sequence of operations to resolve parsing ambiguities based on training data.

❖ **Tree Substitution Grammar (TSG):** TSG is a generative model that generates parse trees by iteratively replacing tree fragments with other trees. TSG can capture various tree structures and is useful for resolving structural ambiguities in parsing.

❖ **Bayesian Grammar Models:** Bayesian grammar models apply Bayesian inference to probabilistic grammars, allowing for uncertainty modeling and ambiguity resolution. These models can provide posterior probabilities for different parse trees, which can be useful in situations where multiple parses have similar probabilities.

Models for Ambiguity Resolution in Parsing

❖ Discriminative models for parsing :

❖ Discriminative models for parsing are a class of models that focus on directly predicting the most likely parse or parsing decisions for a given input sentence without explicitly generating alternative parse trees.

❖ These models are often trained to maximize the likelihood of the correct parse, making them well-suited for structured prediction tasks.

❖ **Conditional Random Fields (CRFs):** Conditional Random Fields are a widely used discriminative modeling technique for various structured prediction tasks, including parsing. In the context of parsing, the input sentence's features (e.g., words, part-of-speech tags) are used to conditionally model the probability of different parse tree structures. CRFs can handle both syntactic and semantic parsing tasks and are known for their strong performance.

Models for Ambiguity Resolution in Parsing

❖ Discriminative models for parsing :

❖ **Maximum Entropy Models (MaxEnt):** Maximum entropy models, often used for part-of-speech tagging and syntactic parsing, assign probabilities to potential parses based on a combination of features derived from the input sentence. These features may include word identities, part-of-speech tags, and contextual information. MaxEnt models aim to maximize the likelihood of the correct parse based on the given features.

❖ **Structured Support Vector Machines (SVMs):** Structured Support Vector Machines are used for structured prediction problems, including parsing. These models seek to find a hyperplane that best separates the correct parse from other possible parses in a high-dimensional feature space. SVMs are effective in learning complex decision boundaries for parsing tasks.

❖ **Transition-Based Parsing Models:** Transition-based parsers are discriminative models that predict parsing actions to build a parse tree incrementally. These models use a set of parsing actions and associated classifiers to make decisions at each step. They are particularly useful for dependency parsing.

